

## Atividade: Policy Gradient com Actor-Critic (A2C) — CartPole-v1

Aluno: Davi de Souza Andrade | Fork com `algorithms/a2c.py` e `networks/discrete_actor_critic.py` completos:  
<https://github.com/DavideSouzaAndrade/t-zero-learning>

**Implementação.** Parte 1: retorno de  $n$  passos calculado de trás para frente,  $R_t = r_t + \gamma(1-d_t)R_{t+1}$ ,  $R_T = V(s_T)$ , por coluna. Parte 2: perda  $-\text{média}(\log \pi(a|s) \cdot A)$  com  $A = (R - V)$ .`detach()` (ou `R.detach()` sem baseline) — o peso é constante para o gradiente do ator, senão a perda do ator puxaria o crítico. Parte 3: `Categorical(logits)`, amostra ou `argmax`, `log_prob` da ação dada, entropia e  $V(s)$ . Os 14 testes de `tests/test_a2c.py` passam. **Protocolo.** 500 mil passos, config `a2c_cartpole + --override`, 2 seeds (1 e 2; linha cheia = s1, tracejada = s2) em *todas* as configurações (20 runs). Os gráficos usam exatamente as métricas que o harness envia ao wandb (mesmos nomes), registradas localmente. Baseline ( $n_{\text{envs}}=8$ ,  $n=5$ ,  $\text{ent}=0,01$ ): avaliação final  $500 \pm 0$  nas duas seeds, retorno médio  $\geq 450$  aos 148–200 mil passos.

### Q1 — Número de atores em paralelo (`num_envs`)

**Previsão.** Com 1 ambiente o lote vem de um único trecho de trajetória, muito correlacionado: espero um `policy_loss` muito mais ruidoso e um aprendizado instável. Com 64 ambientes o gradiente fica suave, mas, como o lote cresce, cabem menos atualizações em 500 mil passos; espero SPS crescendo com `num_envs`.

**Varredura.** `num_envs`  $\in \{1, 8, 64\}$  e um controle `num_envs=1`, `num_steps=40`, que tem o *mesmo* lote (40) e o mesmo número de atualizações (12.500) do baseline: só muda a origem das transições (1 trajetória  $\times$  8 independentes).

| config                         | lote | atualizações | dp policy_loss | retorno final (last100) | eval final | SPS   |
|--------------------------------|------|--------------|----------------|-------------------------|------------|-------|
| <code>num_envs=1</code>        | 5    | 100.000      | 8,4 / 8,8      | 382 / 350               | 459 / 457  | 890   |
| <code>num_envs=1, n=40</code>  | 40   | 12.500       | 10,5 / 8,8     | 395 / 409               | 370 / 286  | 1.320 |
| <code>num_envs=8 (base)</code> | 40   | 12.500       | 1,6 / 1,4      | 466 / 474               | 500 / 500  | 3.770 |
| <code>num_envs=64</code>       | 320  | 1.562        | 0,5 / 0,5      | 153 / 146               | 172 / 158  | 7.100 |

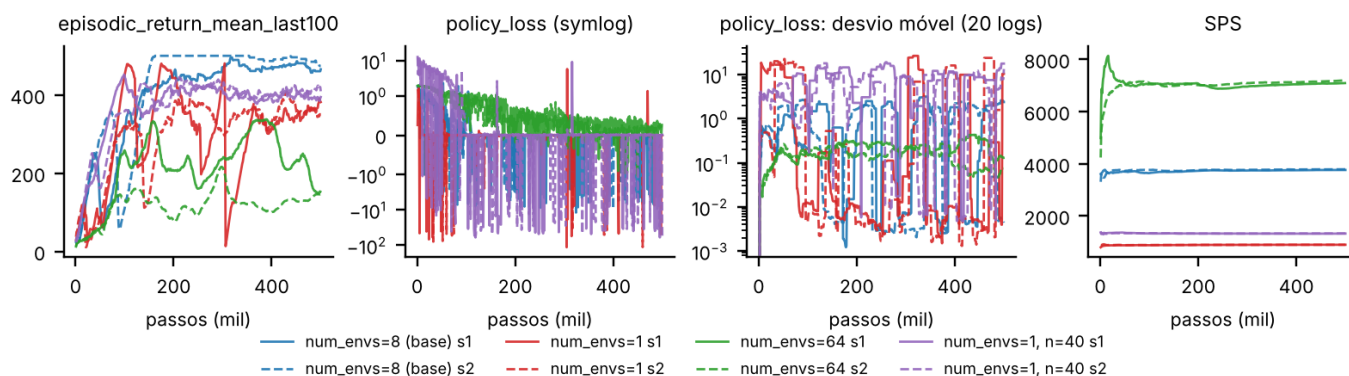


Fig. 1 — Q1. O painel do desvio móvel do `policy_loss` quantifica o ruído do gradiente seguido.

**Explicação.** Há dois efeitos misturados em `num_envs`, e o controle os separa. **(i) Correlação.** Com o mesmo lote e o mesmo número de passos de gradiente do baseline, o controle de 1 ambiente tem um `policy_loss` 6 $\times$  mais ruidoso (desvio de  $\sim 9$ – $10$  contra  $\sim 1,5$ ) e fica preso em  $\sim 400$ , com avaliação de 286–370. Num trecho de 40 passos de uma mesma trajetória os estados são quase iguais e as vantagens têm o mesmo sinal (se o poste está caindo, todas são negativas). Os 40 termos então não se cancelam: o lote é, na prática, uma única amostra do gradiente, enviesada para aquele trecho. Os picos de  $-10$  a  $-100$  no `policy_loss` aparecem quando um episódio termina dentro do lote. Com 8 atores em pontos diferentes de episódios diferentes, a média é de estimativas quase independentes, e a variância cai. É o papel que o replay buffer cumpre no DQN e que um método on-policy não pode usar: esse é o argumento central do A3C, e a previsão se confirmou. **(ii) Tamanho do lote  $\times$  número de atualizações.** `num_envs=64` tem o gradiente mais limpo (desvio de 0,5), mas recebe só 1.562 passos de Adam com  $\text{lr}=7\text{e-}4$  e estaciona em  $\sim 150$ : o limite passa a ser o número de atualizações, não a qualidade de cada uma. Isso eu não previ: esperava apenas "mais lento", e o efeito é forte. Com mais timesteps ou um  $\text{lr}$  maior o resultado tende a mudar. Já `num_envs=1` com  $n=5$  faz 100 mil atualizações minúsculas e ruidosas e ainda chega a eval  $\approx 458$ , mas com quedas bruscas (até  $\sim 20$  de retorno). **(iii) Tempo.** O SPS sobe de 890 ( $\approx 9,5$  min) para 3.770 ( $\approx 2,2$  min) e 7.100 ( $\approx 1,2$  min): um forward da rede sobre o lote de  $N$  observações custa quase o mesmo que sobre 1, então o paralelismo dilui o custo fixo por passo. Ou seja, o paralelismo compra tempo de parede e decorrelação ao mesmo tempo, mas, para um orçamento fixo de passos, `num_envs` grande demais custa atualizações.

### Q2 — Horizonte do retorno de $n$ passos (`a2c.num_steps`)

**Previsão.**  $n=1$  (quase só bootstrap) tem baixa variância e alto viés, porque herda o erro de  $V$ .  $n$  grande (quase Monte Carlo) tem pouco viés e alta variância, então espero `value_loss` maior e `explained_variance` menor; e o ator piora quando o crítico piora. **Varredura.**  $n \in \{1, 5, 32, 128\}$ , com lote de 8 / 40 / 256 / 1.024 e 62.500 / 12.500 / 1.953 / 488 atualizações.

| config                  | value_loss médio | explained_var. média | advantage_mean médio | retorno final | eval final |
|-------------------------|------------------|----------------------|----------------------|---------------|------------|
| <code>n=1</code>        | 22 / 26          | 0,75 / 0,50          | -0,5                 | 488 / 297     | 500 / 455  |
| <code>n=5 (base)</code> | 36 / 32          | 0,14 / 0,49          | -0,5                 | 466 / 474     | 500 / 500  |
| <code>n=32</code>       | 73 / 88          | 0,11 / 0,14          | +5                   | 493 / 265     | 500 / 309  |
| <code>n=128</code>      | 502 / 497        | 0,007 / 0,007        | +26                  | 445 / 333     | 431 / 422  |

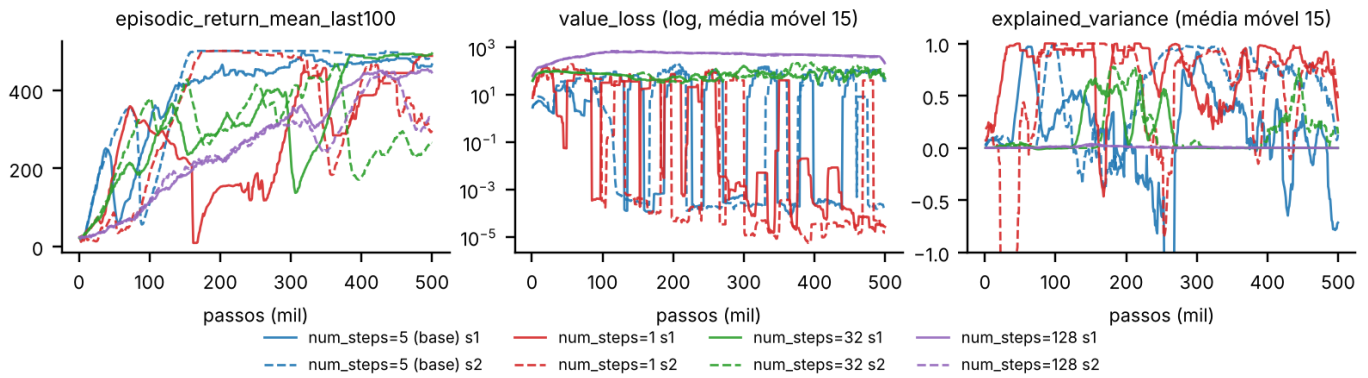


Fig. 2 — Q2. Value\_loss e explained\_variance suavizados por média móvel (15 logs).

**Explicação.** O alvo de  $n=1$ ,  $r + \gamma V(s')$ , contém uma única recompensa amostrada, então tem pouca variância; mas seu erro é o erro de  $V(s')$ , ou seja, viés. Com  $n=128$  o alvo soma até 128 recompensas que dependem de uma política estocástica e do instante em que o poste cai: viés quase nulo e variância grande. Os gráficos confirmam a previsão. Com  $n=128$  o *value\_loss* fica em  $\sim 500$  e a *explained\_variance* em  $\approx 0$  durante todo o treino: o crítico não explica nada. Para isso contribuem os alvos ruidosos e também o fato de ele só receber 488 passos de gradiente (o lote mudou). O *advantage\_mean* de +26 mostra que  $V$  subestima sistematicamente os retornos e está sempre atrasado. Como o ator usa  $A = R - V$ , um crítico que não explica nada deixa  $A \approx R - \text{const}$ : o baseline deixa de reduzir a variância (o mesmo mecanismo da Q4), e o aprendizado fica lento e quase linear. **A armadilha:** com  $n=1$  a *explained\_variance* é a maior da varredura (0,5–0,75), muitas vezes perto de 1. Mas o alvo é  $1 + \gamma V(s')$  e, no CartPole,  $V(s') \approx V(s)$  em estados consecutivos: a variância do alvo é quase toda a variância da própria saída do crítico. A EV alta mede autoconsistência ( $V$  concorda com  $V$  deslocado), não acerto em relação ao retorno verdadeiro; um  $V$  uniformemente errado teria  $EV \approx 1$  do mesmo jeito. Prova disso é a seed 1 de  $n=1$ , que despencou para retorno  $\sim 10$  aos 160 mil passos com EV perto de 1 no mesmo período, e a seed 2, que caiu de 500 para  $\sim 300$  no fim. Observação: depois da convergência (retorno 500) muitos lotes não têm nenhum término, os alvos ficam quase constantes e o *value\_loss* cai a  $\sim 1e-4$  enquanto a EV fica mal definida ( $\text{var}(y) \approx 0$ ); daí as oscilações bimodais em  $n=1$  e  $n=5$ , que não indicam piora do crítico. O compromisso  $n=5$  é o único estável nas duas seeds.

### Q3 — Coeficiente de entropia (a2c.ent\_coef)

**Previsão.** Com *ent\_coef*=0,1 a entropia deve ficar perto de  $\ln 2 \approx 0,69$  e o retorno deve estacionar, porque a política não pode ficar quase determinística. Com *ent\_coef*=0 a entropia cai mais rápido, com risco de colapso prematuro, embora o CartPole talvez tolere. **Varredura.** *ent\_coef*  $\in \{0, 0,01, 0,1\}$ , 2 seeds cada. **Resultado.** *ent*=0: entropia final  $\approx 0,47$ , retorno  $\geq 450$  aos 122 e 141 mil passos, eval 500/500 (o mais rápido). *ent*=0,01: entropia 0,54, eval 500/500. *ent*=0,1: entropia  $\approx 0,60$ , retorno preso em 346–377, eval 390/415.

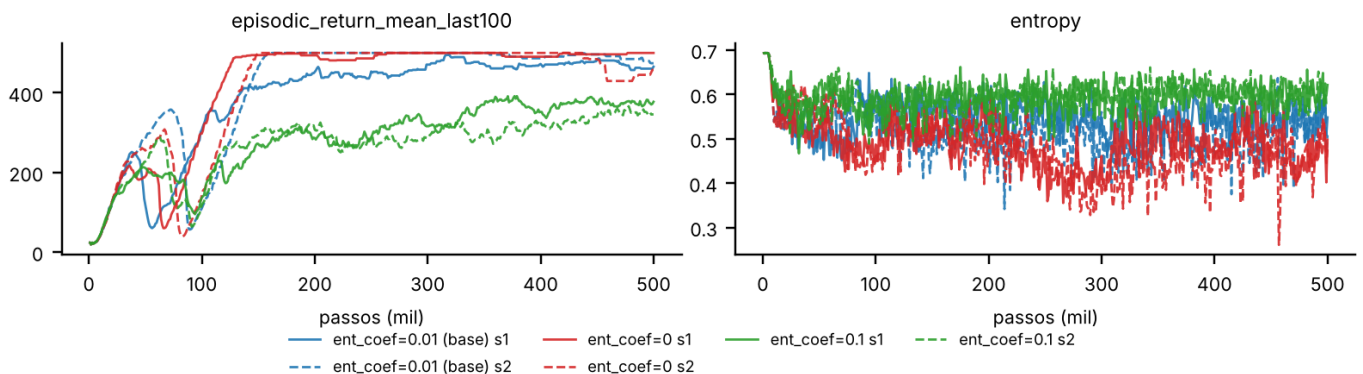


Fig. 3 — Q3. Retorno médio e entropia da política para os três coeficientes.

**Explicação.** O bônus de entropia impede que a política fique determinística cedo demais, antes que o crítico e as vantagens sejam confiáveis. Num método on-policy isso é grave, porque a política só aprende com as ações que ela mesma amostra: se uma ação deixa de ser amostrada, nunca mais recebe gradiente. Com *ent\_coef*=0,1 o termo  $-c \cdot H$  compete com o gradiente de política. Perto de uma boa política as vantagens ficam pequenas ( $|A| \sim 2-3$ ), e então o gradiente da entropia domina e segura a política em  $\sim 60/40$  em vez de  $\sim 95/5$ . As ações "erradas" amostradas de vez em quando derrubam o poste, e o retorno estaciona: a política é ótima para o objetivo regularizado, não para o retorno (previsão confirmada). Com *ent\_coef*=0 o CartPole funcionou nas duas seeds, e até mais rápido. O ambiente é benevolente: são só 2 ações, a recompensa é densa (+1 a cada passo) e uma ação ruim se manifesta em poucos passos. A inicialização do ator (*std* 0,01) começa uniforme e a própria estocasticidade inicial explora tudo o que existe para explorar. Além disso, com baseline as vantagens são centradas em  $-0$  e empurram nos dois sentidos, por isso a entropia não colapsou (0,47; compare com a Q4). Num ambiente com muitas ações ou recompensa rara (p. ex. LunarLander, com o grande bônus terminal), eu esperaria que, sem bônus, a política se fixasse cedo na primeira ação medíocre que dá algum retorno (p. ex. ficar pairando) e nunca descobrisse a recompensa rara; o bônus passa a ser necessário.

### Q4 — Ablção do baseline (a2c.use\_baseline=false)

**Previsão.** Sem baseline o peso passa a ser  $R$ , que é positivo e da ordem de dezenas: *advantage\_mean* muito acima de 0, *advantage\_std* bem maior e treino mais ruidoso. Com pesos sempre positivos, toda ação amostrada é reforçada, e espero que a entropia colapse. **Varredura.** *use\_baseline*  $\in \{\text{true}, \text{false}\}$ , 2 seeds. **Resultado.** Média sobre o treino: *advantage\_mean* de 50–70 (contra  $-0,5$ ), *advantage\_std* de 8,6–14,8 (contra 2,6), entropia de 0,06–0,15 aos 100 mil passos e 0,004 no fim (contra 0,54). Retorno: a seed 1 chega a 330 e degrada para 86 (eval 105); a seed 2 chega a 500 aos 206 mil passos, despencou para  $\sim 150$  aos 290 mil e termina em 354 (eval 480).

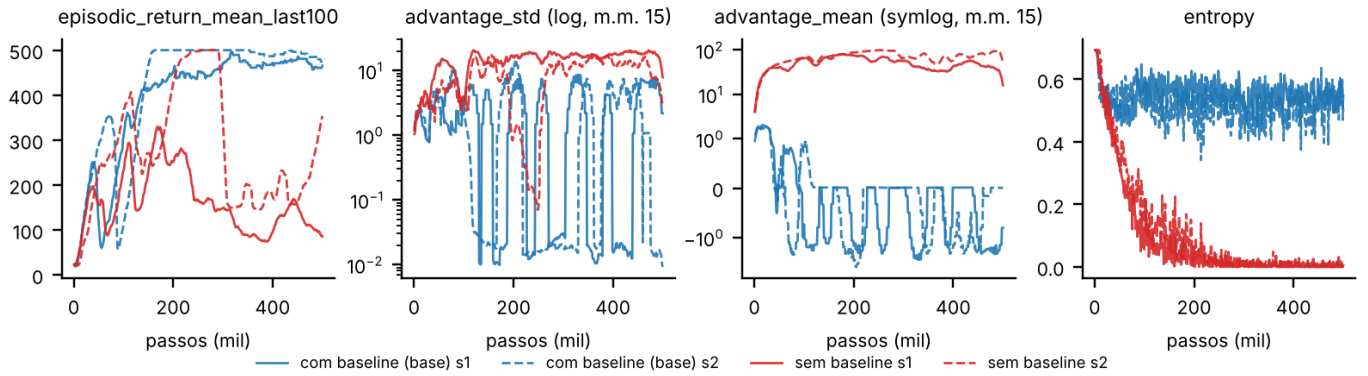


Fig. 4 — Q4.  $\text{advantage}_* = \text{pesos efetivamente usados no gradiente } (R - V \text{ com baseline}; R \text{ sem})$ . Média móvel de 15 logs.

**Explicação.** Como  $\sum_a \nabla \pi(a|s) = 0$ , temos  $E[\nabla \log \pi \cdot b(s)] = 0$ : subtrair  $b(s) = V(s)$  não muda o gradiente esperado, mas o estimador amostral passa de  $\log \pi \cdot R$  para  $\log \pi \cdot (R - V)$ , e sua variância depende do segundo momento dos pesos. Os gráficos de  $\text{advantage}_*$  confirmam isso: com baseline os pesos são centrados (média  $\approx 0$ ) e têm desvio  $\sim 2,6$ ; sem ele, têm média 50–70 e desvio 3–6× maior. A maior parte do peso é um termo comum a todas as ações, que não carrega informação e só acrescenta ruído. A entropia mostra a consequência. No CartPole toda recompensa é +1, logo  $R > 0$  sempre, e cada atualização aumenta  $\log \pi$  de *todas* as ações amostradas. A ação já mais provável é amostrada mais vezes e por isso é mais reforçada ("rich get richer"). Em esperança o termo comum se cancela, mas com lotes finitos ele é um empurrão aleatório e forte rumo ao determinismo, e com pesos  $\sim 50$  o bônus de  $0,01 \cdot H$  é irrelevante. A entropia cai a 0,004 (política determinística) e, como  $\nabla \log \pi \rightarrow 0$  para a ação de probabilidade  $\approx 1$ , o agente não consegue mais explorar nem se corrigir: a seed 1 trava numa política ruim e a seed 2, que tinha chegado a 500, desaba e não se recupera. Previsão confirmada: o baseline é o que torna o gradiente útil, e no CartPole a ausência dele afeta mais a estabilidade (colapso de entropia) do que a velocidade inicial.

Reprodução: `python scripts/run_logged.py logs/X.jsonl --config a2c_cartpole --override seed=S ...` (lista em `scripts/jobs.txt`); gráficos: `python scripts/make_plots.py`.